Iso-distance comparative study of neutron flux in spherical moderators using OpenMC

- **Author :** Clément DAROUSSIN
- **Date :** March 2025
- **Code and version :** OpenMC 0.15.3
- **Datas :** JEFF 3.3 (Continuous energy)

ABSTRACT

This numerical study presents an iso-distance comparative analysis of light water (H_2O), graphite, and heavy water (D_2O) moderation characteristics using Monte Carlo simulations.

The calculated metrics yield 99.98% absorption, $L_{eff} = 3.94cm$, $M.R. \approx 62$, and $0.15ms$ residence time for light water (H_2O); 49.93% absorption, $L_{eff} = 14.99cm$, $M.R. \approx 224$, and $3.14ms$ residence time for graphite; and 10.39% absorption, $L_{eff} = 15.89cm$, $M.R. \approx 5310$, and $4.18ms$ residence time for heavy water (D_2O).

This work thus quantifies the fundamental trade-off in nuclear engineering between the volumetric compactness of light water cores (requiring enriched uranium) and the preservation of neutron economy in heavy water and graphite systems (allowing the use of natural uranium).

Summary :

1. INTRODUCTION AND PARAMETERS JUSTIFICATION..... 2
2. GEOMETRIC JUSTIFICATION..... 2
3. NUCLEAR DATA..... 3
4. MATERIAL DEFINITIONS AND NUCLEAR PROPERTIES..... 4
5. GEOMETRY..... 5
6. SOURCE DEFINITION..... 6
7. SETTINGS..... 6
8. MESHING..... 6
9. TALLIES..... 6
10. RUNNING OPENMC..... 8
11. POST-PROCESSING..... 9
12. FINAL CONCLUSION AND ENGINEERING PERSPECTIVES..... 21

1. INTRODUCTION AND PARAMETERS JUSTIFICATION

The simulation is performed using the OpenMC Monte Carlo code based on a pseudo-random number generator. The reliability of our results is based on two theoretical pillars :

- **Law of large numbers :** Each particle is simulated as an independent and identically distributed event, the statistical error on estimators then decreases proportionally to the inverse of the square root of the number of histories regardless of the dimensionality of the sampled space. For this preliminary study, we aim for a relative error of approximately $\sigma_{rel} = 0{,}03\%$ **(0, 0003)** giving :

$$\sigma_{rel} = \frac{\sigma_{\bar{x}}}{\mu} = \frac{\sigma}{\mu\sqrt{N}} \qquad (1)$$

Since neutron transport follows a Poisson distribution, the intrinsic relative uncertainty of a single particle history is 100% ($\sigma/\mu = 1$) :

$$\sigma_{rel} = \frac{1}{\sqrt{N}} \implies \sqrt{N} = \frac{1}{0{,}0003} \implies N = 11\,111\,111 \quad (2)$$

- **Batching strategy :** We'll calculate 10 million histories for this calculation, rather than processing particles in a single block, we divide them into batches. According to the central limit theorem, the distribution of the means of each batch tends toward a normal distribution. This allows for the calculation of a robust variance and a realistic confidence interval for each score (tally). Selected parameters are 10 batches of 1,000,000 particles (utilizing 10 batches of 1M particles, rather than 100 batches of 100k is preferred to optimize memory usage and prevent RAM saturation).

2. GEOMETRIC JUSTIFICATION

```python
import matplotlib.pyplot as plt
import numpy as np

# Data
radii = np.array([50, 75, 100, 125, 150, 175, 200])
# 1e-5 pour représenter le "0" en log
leakage_h2o = np.array([0.24, 0.02, 1e-5, 1e-5, 1e-5, 1e-5, 1e-5])
leakage_graphite = np.array([94.09, 80.66, 64.80, 50.17, 37.38, 27.22, 19.32])
leakage_d2o = np.array([98.82, 96.62, 93.72, 90.08, 86.11, 81.74, 76.89])

plt.figure(figsize=(10, 6))

# Plotting lines
plt.plot(radii, leakage_h2o, 'o-', label='Light Water (H2O)', color='royalblue', lw=2)
plt.plot(radii, leakage_graphite, 's-', label='Graphite', color='dimgray', lw=2)
plt.plot(radii, leakage_d2o, '^-', label='Heavy Water (D2O)', color='lightseagreen', lw=2)
plt.axvline(x=125, color='red', linestyle='--', lw=1, label='Analysis Threshold (125 cm)')

# Formatting
plt.xlabel('Sphere Radius (cm)', fontsize=12)
plt.ylabel('Neutron Leakage (%)', fontsize=12)
plt.title('Neutron Leakage vs. Shielding Thickness', fontsize=14)
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.legend()

# Adding an annotation for the "0%" Water
plt.annotate('Total Shielding', xy=(100, 1e-5), xytext=(70, 20),
             arrowprops=dict(facecolor='black', shrink=0.05, width=1))

plt.tight_layout()
plt.show()
```

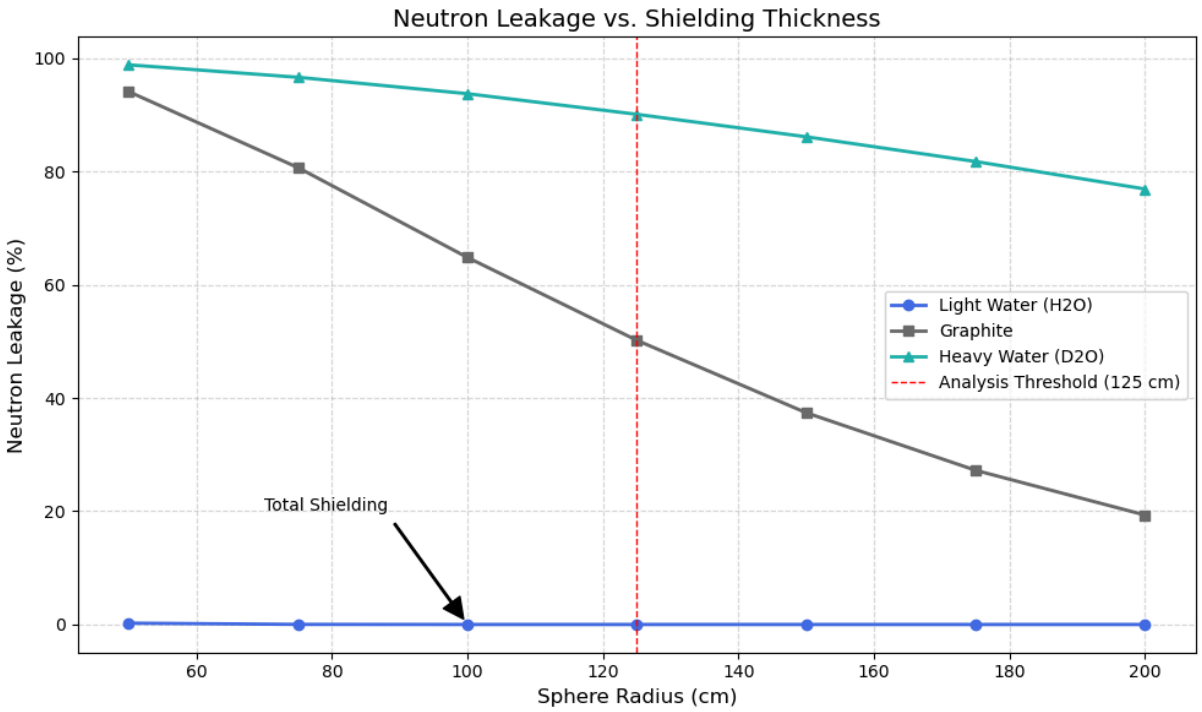


Figure. 1 - Sensitivity analysis of neutron leakage vs. shielding thickness

Following a quick sensitivity analysis of neutron leakage across various dimensions ($r \in [50cm, 200cm]$), it was concluded that the 125 cm radius represents the most effective scale for this comparative study, providing the "ideal window" to observe the diverging physics of the three selected materials because it offers the optimal contrast threshold. At this radius :

- **Light water :** reaches total extinction (shielding regime).
- **Graphite :** reaches its relaxation length. It serves as the perfect median pivot point, showing half of the neutron population being attenuated.
- **Heavy water :** remains near-transparent (diffusion regime).

This sensitivity analysis showed that outside of this specific radius, the results tend to converge (for graphite and heavy water below and for graphite and light water above). This choice makes the physical concept of migration length much easier to interpret in the final results.

3. NUCLEAR DATA

The calculation relies on the H5 continuous-energy at 293.6 K nuclear data libraries from the JEFF-3.3-HDF5 library containing the precise cross-sections required to accurately model scattering phenomena and thermal capture.

```
import openmc
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as colors
from tqdm.auto import tqdm
import time

#cleaning
!rm -r *h5

# Study Parameters
sphere_radius = 125 # in cm
nb_particles = 1000000
nb_batches = 10
mesh_size = 300
```

4. MATERIAL DEFINITIONS AND NUCLEAR PROPERTIES

In this section, we define the three media used for the comparative study by three key properties :

- **Isotopic composition :** The precise atomic ratios (e.g., $H/O = 2$) which dictate the probability of specific neutron interactions.
- **Mass density :** This determines the macroscopic cross-section (Σ) and thus the mean free path of neutrons.
- **Thermal scattering data ($S(\alpha, \beta)$) :** Essential for low-energy neutrons, these laws account for the chemical bonds of the atoms (e.g., `c_H_in_H2O`, `c_D_in_D2O`, `c_Graphite`), which significantly affect the final thermalization profile.

```
mats = openmc.Materials()

# 1. Water (H2O)
water = openmc.Material(material_id=1, name="light_water")
water.add_nuclide('H1', 2.0)
water.add_nuclide('O16', 1.0)
water.set_density('g/cm3', 1.0)
water.add_s_alpha_beta('c_H_in_H2O')

# 2. Heavy Water (D2O)
heavy_water = openmc.Material(material_id=2, name="heavy_water")
heavy_water.add_nuclide('H2', 2.0)
heavy_water.add_nuclide('O16', 1.0)
heavy_water.set_density('g/cm3', 1.1) # Densité plus élevée que H2O
heavy_water.add_s_alpha_beta('c_D_in_D2O')

# 3. Graphite (Nuclear Grade)
graphite = openmc.Material(material_id=3, name="graphite")
graphite.add_element('C', 1.0)
graphite.set_density('g/cm3', 1.7) # Densité typique du graphite nucléaire
graphite.add_s_alpha_beta('c_Graphite')


# adding and exporting material
mats.append(water)
mats.append(heavy_water)
mats.append(graphite)

mats.export_to_xml()
```

5. GEOMETRY

5.1. Definition

The model is a simple sphere of radius 125 cm with a vacuum boundary condition, ensuring that any neutron leaving the surface is counted as "leakage".

```
# Surfaces
external_surface = openmc.Sphere(r=sphere_radius, boundary_type='vacuum')

# Cells
cell = openmc.Cell(fill=water, region=-external_surface)

#universe
univ = openmc.Universe(cells=[cell])

#final geometry
geom = openmc.Geometry(univ)
geom.export_to_xml()
```

5.2. Visualization

```
fig, axes_list = plt.subplots(1, 3, figsize=(18, 6))

plot_configs = [
    (water, 'Light Water', 'royalblue'),
    (heavy_water, 'Heavy Water', 'lightseagreen'),
    (graphite, 'Graphite', 'dimgray')]

for i, (mat, name, color_code) in enumerate(plot_configs):
    ax = axes_list[i]
    cell.fill = mat

    # Visualizing the geometry slice
    univ.plot(width=(sphere_radius * 2.2, sphere_radius * 2.2),
              basis='xy',
              color_by='material',
              colors={mat: color_code},
              axes=ax,
              pixels=1000000)

    # Marking the point source at the center
    ax.scatter(0, 0, color='red', s=20, label='Source', zorder=5)

    # Adding radius annotation for scale clarity
    ax.annotate('', xy=(sphere_radius*0.98, 0), xytext=(sphere_radius*0.02, 0),
         arrowprops=dict(arrowstyle='<->', color='Black', lw=1.5))
    ax.text(sphere_radius/2, 8, f'R = {sphere_radius} cm',
                  color='Black', fontsize=10, ha='center', fontweight='bold')

    ax.set_title(f"{name}")

plt.tight_layout()
plt.show()
```

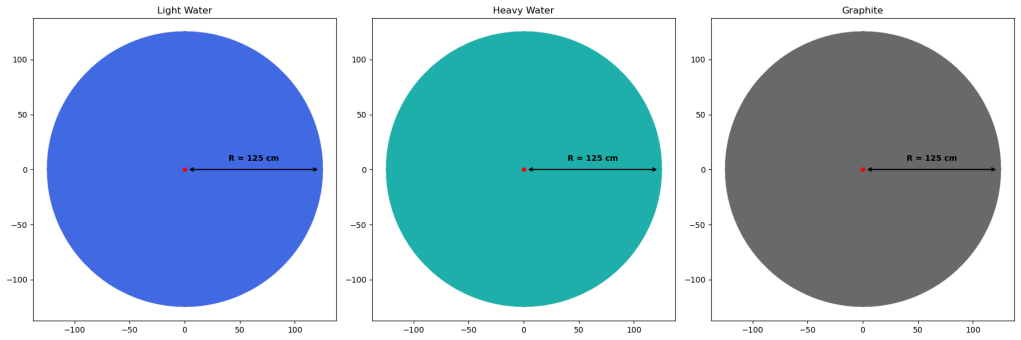


Figure 2. - Geometry visualization ($R = 125cm$)

6. SOURCE DEFINITION

```
source = openmc.IndependentSource()
source.space = openmc.stats.Point((0, 0, 0))
source.energy = openmc.stats.Watt()
```

7. SETTINGS

```
sett = openmc.Settings()
sett.source = source
sett.particles = nb_particles
sett.batches = nb_batches
sett.run_mode = 'fixed source'

sett.export_to_xml()
```

8. MESHING

A 2D regular mesh of cells is defined to discretize the sphere's cross-section.

```
# Create mesh
mesh = openmc.RegularMesh()
mesh.dimension = [mesh_size, mesh_size, 1]
mesh.lower_left = [-sphere_radius, -sphere_radius, -1]
mesh.upper_right = [sphere_radius, sphere_radius, 1]

# Create filter (average in every mesh not only a global one)
mesh_filter = openmc.MeshFilter(mesh)
```

9. TALLIES

The tallies are set up to track the neutron's journey from birth to escape/absorption.

- The **flux spectrum** and **spatial flux** work together. One tells us if the neutrons are actually slowing down (thermalizing), while the other shows how far they travel before doing so. This is how we visualize the "moderation zone."
- The **leakage current** at the 125 cm boundary is used to quantify the number of neutrons (useful neutrons regarding fission) that arrive thermalized in the fuel.
- The **moderation time** adds the final dimension by showing how fast moderation happens. A long moderation time increases the risk of capture, even if the final spectrum looks good.

In short, these tallies allow us to distinguish a shield (low leakage, fast extinction) from a transparent medium (high leakage, fast neutrons) or a true moderator (high leakage, thermalized neutrons).

```
tallies = openmc.Tallies()

# 1. Energy Spectrum Tally
energies = np.logspace(-5, 7, 501)
e_filter = openmc.EnergyFilter(energies)
spec_tally = openmc.Tally(name='flux_spectrum')
spec_tally.filters = [e_filter]
spec_tally.scores = ['flux']
tallies.append(spec_tally)

# 2. Spatial Flux Tally
mesh_tally = openmc.Tally(name='spatial_flux')
mesh_tally.filters = [mesh_filter]
mesh_tally.scores = ['flux']
tallies.append(mesh_tally)

# 3. Leakage Tally
# Requires a surface filter on the outer boundary of your sphere
surf_filter = openmc.SurfaceFilter([external_surface])
leak_tally = openmc.Tally(name='leakage_current')
leak_tally.filters = [surf_filter]
leak_tally.scores = ['current']
tallies.append(leak_tally)

# 4. Moderation Time Tally
# Tracks how long neutrons stay in the system
times = np.logspace(-9, -2, 101) # From nano-s to centi-s
t_filter = openmc.TimeFilter(times)
time_tally = openmc.Tally(name='moderation_time')
time_tally.filters = [t_filter]
time_tally.scores = ['flux']
tallies.append(time_tally)

# 5. Absorption Rate Tally
# This tracks the physical disappearance of neutrons by capture/absorption
abs_tally = openmc.Tally(name='absorption_rate')
abs_tally.scores = ['absorption']
tallies.append(abs_tally)

tallies.export_to_xml()
```

10. RUNNING OPENMC

The three materials are processed through an automated loop. For each material, the geometry is updated, the transport simulation is executed, and the results are harvested directly from the OpenMC statepoint files.

```
material_list = [(water, 'light_water'), (graphite, 'graphite'), (heavy_water, 'heavy_water')]

# Initialize data containers for post-processing
results_2d = {}
results_spectrum = {}
results_time = {}
results_leakage = {}
results_abs = {}

for mat_obj, name in material_list:
    start_time = time.time()
    print(f"\n{'='*60}")
    print(f"{name.upper()}")
    print(f"Configuration: {nb_particles} particles | {nb_batches} batches")
    print(f"{'='*60}")

    # Update geometry and export
    cell.fill = mat_obj
    geom.export_to_xml()

    openmc.run(output=False)

    # Access the statepoint file to harvest tally results
    statepoint_file = f'statepoint.{nb_batches}.h5'
    with openmc.StatePoint(statepoint_file) as sp:
        # 1. SPATIAL FLUX (2D)
        # Reshape the flat array into the spatial mesh grid
        raw_data = sp.get_tally(name='spatial_flux').get_values(scores=['flux'])
        results_2d[name] = raw_data.reshape((mesh_size, mesh_size)).copy()

        # 2. ENERGY SPECTRUM
        # Capture the mean flux values across the defined energy bins
        spec_data = sp.get_tally(name='flux_spectrum').mean.flatten()
        results_spectrum[name] = spec_data.copy()
        # 3. MODERATION TIME
        # Harvest the flux distribution as a function of neutron 'time-of-flight'
        time_data = sp.get_tally(name='moderation_time').mean.flatten()
        results_time[name] = time_data.copy()

        # 4. NEUTRON LEAKAGE AND ABSORPTION
        # Extract the global integral values for the final balance sheet
        results_leakage[name] = sp.get_tally(name='leakage_current').mean.sum()
        results_abs[name] = sp.get_tally(name='absorption_rate').mean.sum()

        elapsed_time = time.time() - start_time
        print(f"✅ {name.upper()} processed in {elapsed_time:.2f} s")

print('')
print("--- Simulation Sequence Complete, data Saved ---")
```

11. POST-PROCESSING

11.1. Spectrum comparison

This section visualizes the neutron flux distribution across the 3 different materials (H_2O, D_2O and graphite). To compare these materials effectively, we'll transform the raw flux into a "lethargy-normalized flux". Neutron energies span over nine orders of magnitude so on a linear energy scale, the thermal peak would be compressed into a nearly invisible vertical line.

A neutron does not lose its energy linearly, but rather by a constant fraction of its energy with each collision. This is why we use lethargy, defined as $u = \ln\left(\frac{E_0}{E}\right) \quad \text{avec } E_0 = 20 \text{ MeV}$, which is in a way, “a measure of the age of the neutron as it undergoes moderation" (P. REUSS). By imposing the conservation of the neutron population, we can write :

$$dn = \phi(u)\,du = |\Phi(E)\,dE| \text{ and since } |du| = \frac{dE}{E}$$

$$\implies \phi(u) \cdot \frac{dE}{E} = \Phi(E)\,dE \implies \phi(u) = E \cdot \Phi(E) \quad (3)$$

In the epithermal range, the flux density follows a $1/E$ distribution. By plotting the lethargy flux $\phi(u)$ (or $E\phi(E)$) on a logarithmic energy axis flattens into a horizontal line making it easier to spot physical deviations or resonances.

```
# Create a figure with two vertical subplots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10), sharex=True)
# Calculate geometric centers and bin widths
energy_centers = np.sqrt(energies[:-1] * energies[1:])
bin_widths = np.diff(energies)
for mat_obj, name in material_list:
    flux_vals = results_spectrum[name]
    # Calculate the standard flux density Phi(E)
    phi_e = flux_vals / bin_widths
    # 1. First plot: Standard Flux Phi(E)
    # Using log-log because Phi(E) spans several orders of magnitude
    ax1.loglog(energy_centers, phi_e, label=name, lw=1)
    # 2. Second plot: Lethargy Flux E * Phi(E)
    ax2.loglog(energy_centers, phi_e * energy_centers, label=name, lw=1)
    # Configuration for the first plot (Standard)
    ax1.set_ylabel(r'Flux Density [$\Phi(E)$]')
    ax1.set_title('Standard Energy Spectrum (1/E Slope)')
    ax1.grid(True, which="both", ls="-", alpha=0.5)
ax1.legend()
# Configuration for the second plot (Lethargy)
ax2.set_xlabel('Energy [eV]')
12
ax2.set_ylabel(r'Lethargy Flux [$E \cdot \Phi(E)$]')
ax2.set_title('Lethargy-Normalized Spectrum (Horizontal Plateau)')
ax2.grid(True, which="both", ls="-", alpha=0.5)
# Thermal reference line on both plots
for ax in [ax1, ax2]:
    ax.axvline(x=0.0253, color='grey', linestyle='--', alpha=0.7, label='0.025 eV (Thermal)')
plt.tight_layout()
plt.show()
```

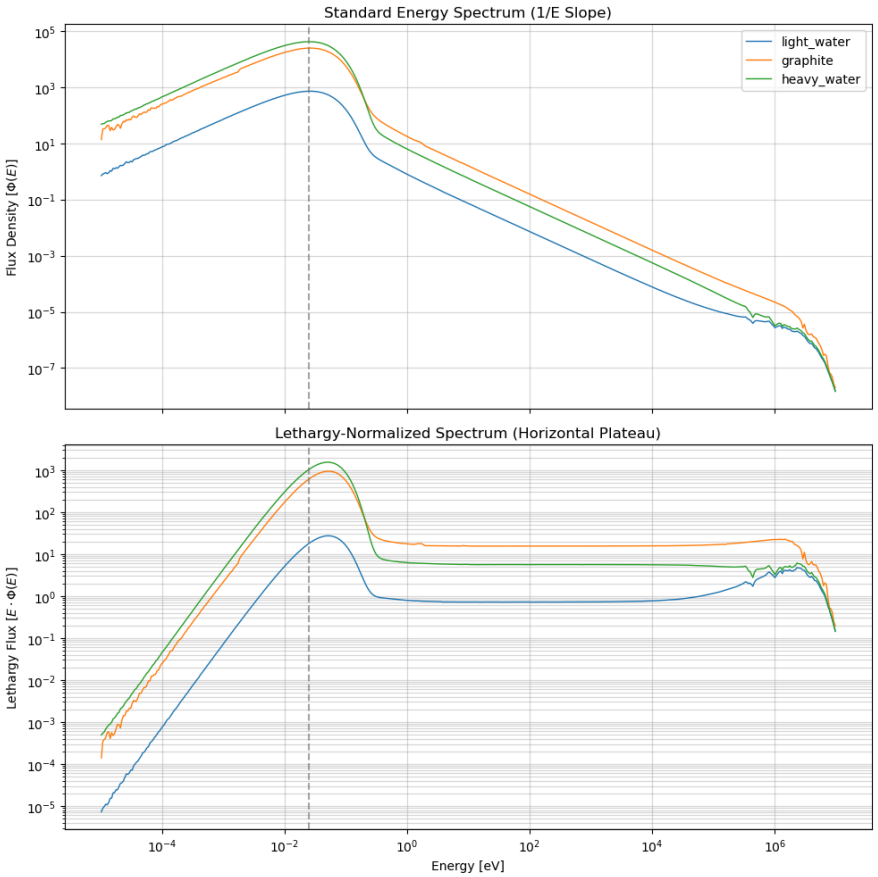


Figure 3. - Comparison between standard and lethargy-normalized spectrum

The script processes the tally results using the following steps :

- **Geometric bin centers :** Calculated as $E_{\text{center},i} = \sqrt{E_i \cdot E_{i+1}}$. (the precise midpoint on a logarithmic axis).
- **Flux density normalization :** Raw tally results are normalized by the bin width (ΔE) to compute the flux density .
- **Thermal reference :** A vertical marker is placed at 0.0253 eV, representing the most probable energy of neutrons in thermal equilibrium at 20°C.

```
plt.figure(figsize=(15, 9))

# Geometric bin centers
energy_centers = np.sqrt(energies[:-1] * energies[1:])
bin_widths = np.diff(energies)
peaks=[]
#printing maxes tab
print(f"{'Material':<12} | {'E_peak (eV)':<12} | {'Flux_max'}")
for mat_obj, name in material_list:
    y = (results_spectrum[name] / bin_widths) * energy_centers
    idx = np.argmax(y[energy_centers < 1])
    peaks.append(y[idx])
    print(f"{name:<12} | {energy_centers[idx]:<12.4f} | {y[idx]:.2e}")
e_max_list = [energy_centers[np.argmax((results_spectrum[m[1]]/bin_widths)*energy_centers *
(energy_centers < 1))] for m in material_list]
e_peak=np.mean(e_max_list)
```

```
print(f"Mean of peaks : {e_peak:.4f} eV")

i=0
for mat_obj, name in material_list:
    # Retrieve tally results for the specific material
    flux_vals = results_spectrum[name]
    # Flux density conversion
    plt.loglog(energy_centers, (flux_vals / bin_widths) * energy_centers, label=name, lw=1)
    plt.scatter(e_peak, peaks[i],marker='+', color='Black', s=30, zorder=5)
    plt.text(e_peak * 1.1, peaks[i], f"{peaks[i]:.3f} eV",
     fontsize=10,verticalalignment='center')
   i+=1

# Thermal reference line
plt.grid(True, which="both", ls="-", alpha=0.5)
plt.minorticks_on()
plt.xlabel('Energy [eV]')
plt.ylabel(r'Lethargy Flux [E(E)]')
plt.title('Energy Spectrum Comparison: Thermalization Profile')
plt.axvline(x=0.0253, color='grey', linestyle='--', alpha=0.7, label='0.025 eV (Thermal)')
plt.axvline(x=e_peak, color='red', linestyle='--', alpha=0.7, label='E of peaks')
plt.legend()
plt.show()
```

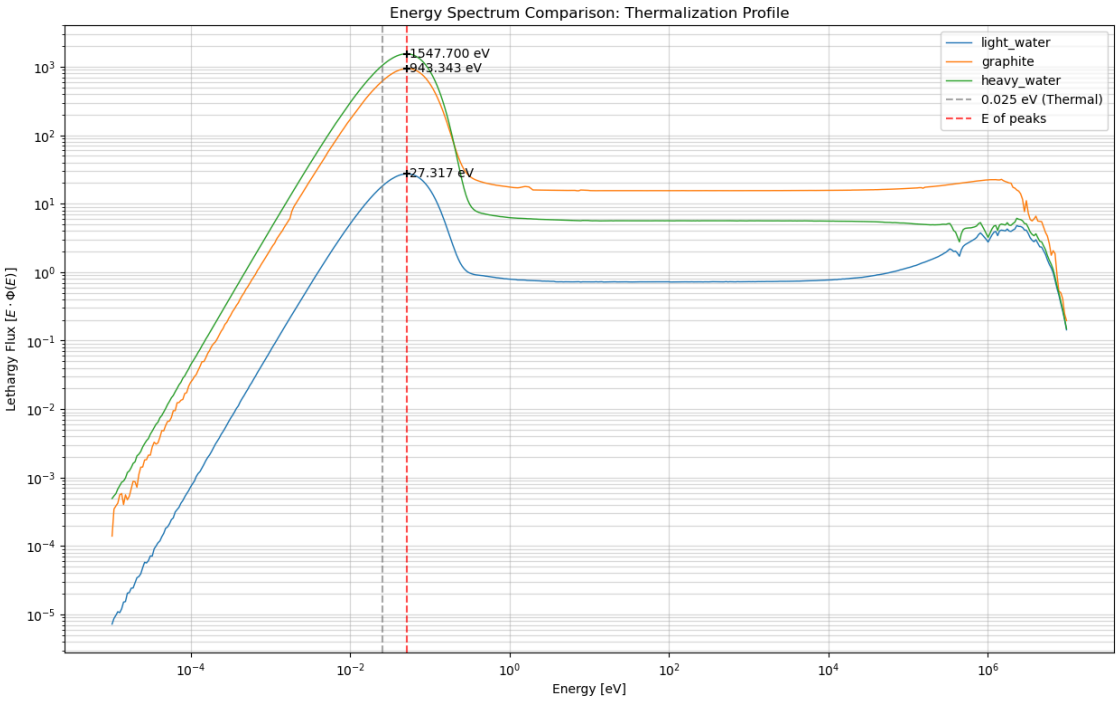


Figure 4. - Energy spectrum comparison

Material	E_{peak} (eV)	Max Flux
$H2O$	0.0511	2.73e+01
Graphite	0.0511	9.43e+02
$D2O$	0.0511	1.55e+03

Table. 1 - Thermal energy peaks (E_{peak}) and maximum flux density

11.2. Moderation time

It is the duration between a neutron's birth and the moment it reaches a specific energy threshold (thermal for moderators study) as we previously saw it is essential for moderator characterization. Just as with the energy spectrum, the script processes the temporal data to ensure accurate visualization through geometric centering and a log-log representation. It allows the simultaneous observation of the slowing-down phase (typically appearing as a plateau or a steady slope) and the thermalization and decay phase (the final roll-off of the curve).

```
plt.figure(figsize=(15, 9))

# Loop on materials
for name in results_spectrum.keys():
    time_vals = results_time[name]

    # Geometric centers
    time_centers = np.sqrt(times[:-1] * times[1:])

    # Log-log representation
    plt.loglog(time_centers, time_vals, label=name, lw=1)

plt.xlabel('Time [s]')
plt.ylabel('Flux Intensity')
plt.title('Neutron Slowing-Down Time Distribution')
plt.grid(True, which="both", ls="-", alpha=0.4)
plt.legend()
plt.show()
```

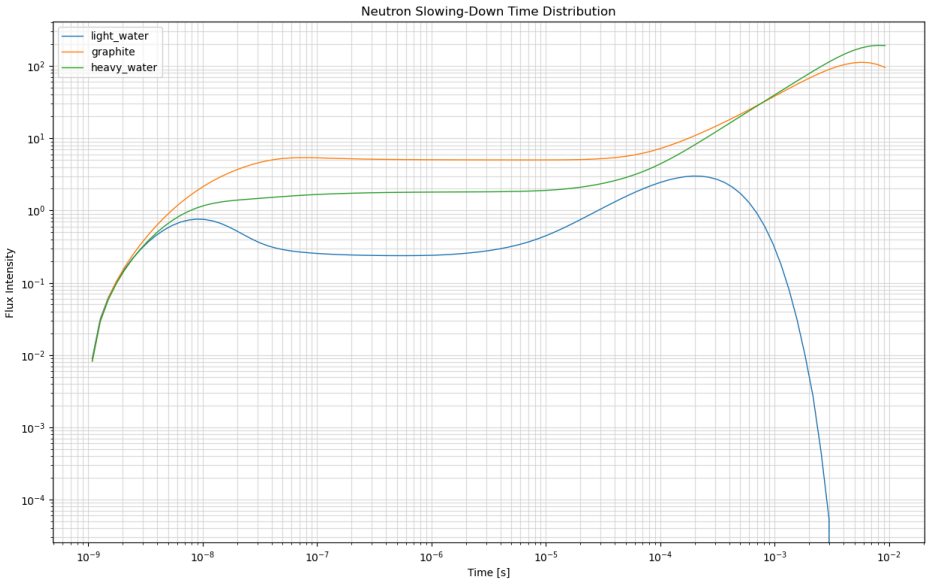


Figure 5. - Neutron slowing-down time distribution

Recalling that the neutron flux is expressed as $\phi = nv$ we can interpret the curves :

Light water (H_2O) : The first peak represents neutrons that thermalize almost instantly (only a few collisions needed). The plateau and second peak show a rapid buildup of thermal neutrons. Once thermalized, neutrons are quickly absorbed making this sharp decay (due to high hydrogen capture).

Heavy water (D_2O) & graphite : The steeper slope and the rise toward a higher plateau show that these moderators act more as "accumulators. The direct and higher plateau proves that the thermal neutron population is significantly larger than in light water. Since there is almost no capture, the flux builds up until it reaches a much higher steady-state (the plateau). However, the graphite plateau reaches a higher magnitude than that of heavy water due to its significantly higher atomic density and macroscopic scattering cross-section (Σ_s).

This tells you how long the flux remains usable after a neutron pulse; we can conclude that water is a fast-impulse response moderator, while heavy water and graphite exhibit high neutronic inertia, favoring a sustained thermal flux over time. The solid graphite lattice acts as a superior spatial trap than, confining the thermal neutron population within a smaller volume and thus increasing the local flux density..

11.3. Spatial distribution

11.3.a - 1D-heatmap

The 1d heatmap visually captures neutron slowing-down and attenuation as a function of distance through a color gradient. It enables a quick qualitative comparison between materials, allowing their efficiency and penetration depth to be ranked at a single glance.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm

# Calculate global limits to ensure all materials use the same color scale
all_data = np.concatenate([d.flatten() for d in results_2d.values()])
vmax = all_data.max()
vmin = vmax / 1e5  # Show 5 orders of magnitude

fig, axs = plt.subplots(len(results_2d), 1, figsize=(10, 2 * len(results_2d)))

if len(results_2d) == 1: axs = [axs]

for i, (name, data) in enumerate(results_2d.items()):
    # Extract radial slice from the horizontal center
    radial_slice = data[mesh_size // 2, :]
    band = radial_slice.reshape(1, -1)

    # Plot using global vmin/vmax for direct comparison
    im = axs[i].imshow(band, cmap='viridis', aspect='auto',
                       extent=[-sphere_radius, sphere_radius, 0, 0.1],
                       norm=LogNorm(vmin=vmin, vmax=vmax))
    axs[i].set_title(f"Material: {name}")
    axs[i].set_yticks([])
    plt.colorbar(im, ax=axs[i], pad=0.01, label='Flux Intensity')

plt.xlabel('Distance from center [cm]')
plt.tight_layout()
plt.show()
```

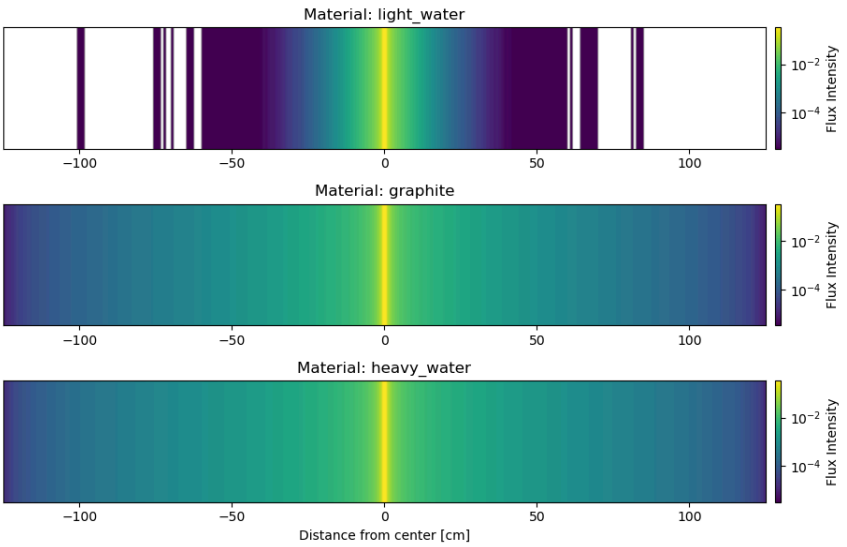


Figure 6. - 1D thermal heatmap of neutron flux intensity

While water acts as a powerful shield due to its high absorption, *D2O* and graphite behave as efficient diffusers demonstrating the difference in their respective diffusion lengths. It allows us to spatially visualize what was discussed previously:

- Light water chokes and absorbs the neutron flux at a short distance from the center.
- Graphite and heavy water allow neutrons to diffuse and carry the flux much further and much more homogeneously throughout the sphere.

11.3.b - 1D-radial flux profile

Even though this 1D graph is very nice and expressive, we need to quantify the attenuation slope (directly related to the material's total cross-section and its ability to stop or scatter neutrons) and the value of flux at any given distance.

```
# Define material colors for consistent plotting
material_colors = {
    'light_water': 'royalblue',
    'heavy_water': 'lightseagreen',
    'graphite': 'dimgray'
    }
plt.figure(figsize=(15, 9))
print(f"{'Material':<15} | {'Slope':<10} | {'L (cm)':<10}")
print("-" * 40)
for name, data in results_2d.items():
    mid = mesh_size // 2
    radial_slice = data[mid, mid:]
    x_axis = np.linspace(0, sphere_radius, len(radial_slice))
    color = material_colors.get(name, 'black')
    # slope calculation
    fit_mask = (x_axis >= 5) & (x_axis <= 30)
    slope, _ = np.polyfit(x_axis[fit_mask], np.log(radial_slice[fit_mask] + 1e-15), 1)
    L = -1/slope

    # Forced Intercept
    intercept_max = np.log(radial_slice[0] + 1e-15)

    print(f"{name:<15} | {slope:<10.4f} | {L:.2f}")

    #Plotting
    plt.semilogy(x_axis, radial_slice,
                 label=f'{name} (Slope={slope:<10.4f}; L={L:.2f})',
                 color=color, lw=1.5)

    # Pure exponential dashed line
    plt.semilogy(x_axis, np.exp(slope * x_axis + intercept_max),
                 color=color, ls='--', lw=1, alpha=0.4)
# Formatting
plt.xlabel('Distance from Center [cm]')
plt.ylabel('Neutron Flux (Log Scale)')
plt.title('1D Radial Flux Profile: Comparison with Pure Exponential Decay')
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.legend()
plt.ylim(1e-6, 1.0) # Keeps the plot clean
plt.xlim(0, 125) # Keeps the plot clean
plt.show()
```

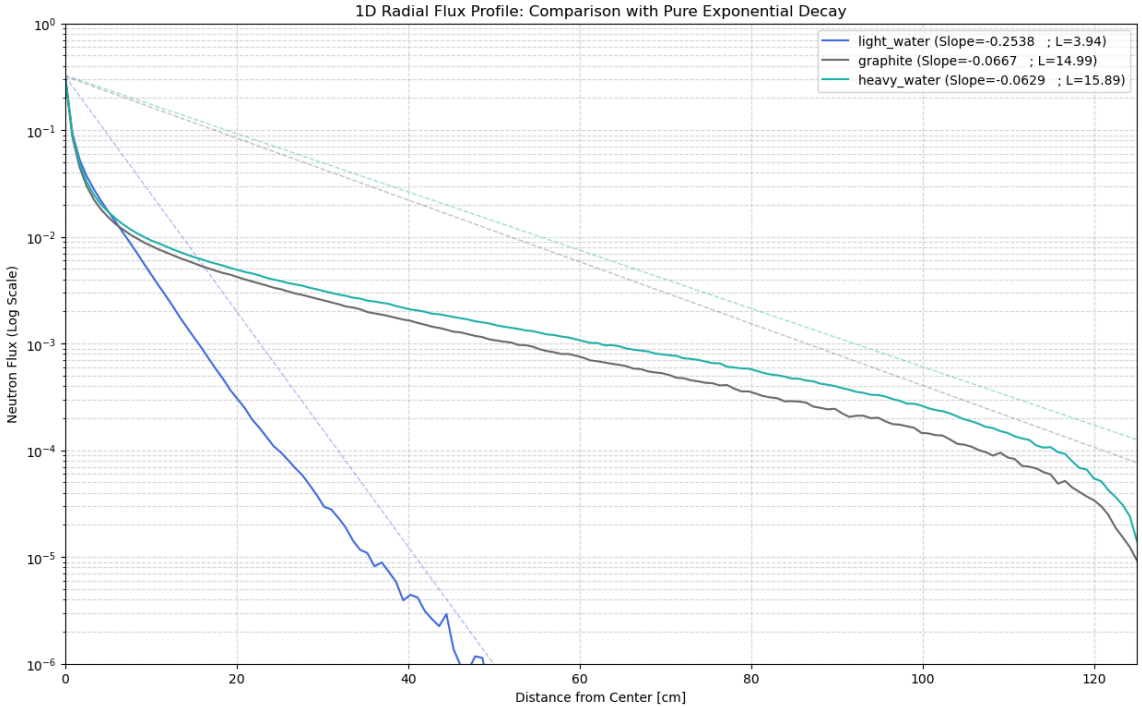


Figure 7. - 1D radial flux profile

11.3.c - Final Comparative Analysis & Statistical Convergence

On a semi-logarithmic plot, the slope describes how rapidly the neutron population decreases per unit distance as the neutrons diffuse and migrate away from the source through the moderating medium.

Material	**Slope (α)**	L_{eff}**(cm)**	**Physical Significance**
H2O	-0.2538	3.94	**Prompt Filter :** High absorption kills the population quickly.
Graphite	-0.0667	14.99	**Spatial Trap :** High density confines neutrons (moderate attenuation).
D2O	-0.0629	15.89	**Accumulator :** Lowest gradient, allowing maximum flux propagation.

Table. 2 - Spatial flux density attenuation characteristics

Light water is characterized by severe spatial attenuation, confirming strong absorption that drastically confines neutrons to short distances. In contrast, graphite and heavy water exhibit much more extended diffusion properties, with heavy water offering the lowest attenuation and optimal penetration of thermal neutrons across the entire system.

11.4. 2D-spatial flux distribution

This visualization provides a cross-sectional map of the neutron flux; it is mathematically redundant in a spherical geometry where the flux depends only on the radial dimension but useful for all other geometries. It allows for an intuitive observation of how the flux attenuation through the material.

```
fig, axes = plt.subplots(1, 3, figsize=(27, 9))

# Establish a common normalization scale for consistent visual comparison
vmax_common = max([res.max() for res in results_2d.values()])

# Loop on all materials
for i, (mat_obj, name) in enumerate(material_list):
    ax = axes[i]
    data_plot = results_2d[name]

    # Render spatial distribution using logarithmic scaling
    im = ax.imshow(data_plot,
                   extent=[-sphere_radius, sphere_radius, -sphere_radius, sphere_radius],
                   norm=colors.LogNorm(vmin=1e-6, vmax=vmax_common),
                   cmap='viridis',
                   origin='lower',
                   interpolation='none')

    # Overlay showing boundary of the geometry
    theta = np.linspace(0, 2*np.pi, 100)
    ax.plot(sphere_radius*np.cos(theta), sphere_radius*np.sin(theta), color='red',
    linewidth=1.5)

    # Configuration of limits and titles
    ax.set_xlim(-sphere_radius*1.05, sphere_radius*1.1)
    ax.set_ylim(-sphere_radius*1.05, sphere_radius*1.1)
    ax.set_title(f'{name}', fontsize=25)

    # Colorbar
    plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04, label='Flux [n/cm²-s]')

plt.tight_layout()
plt.show()
```

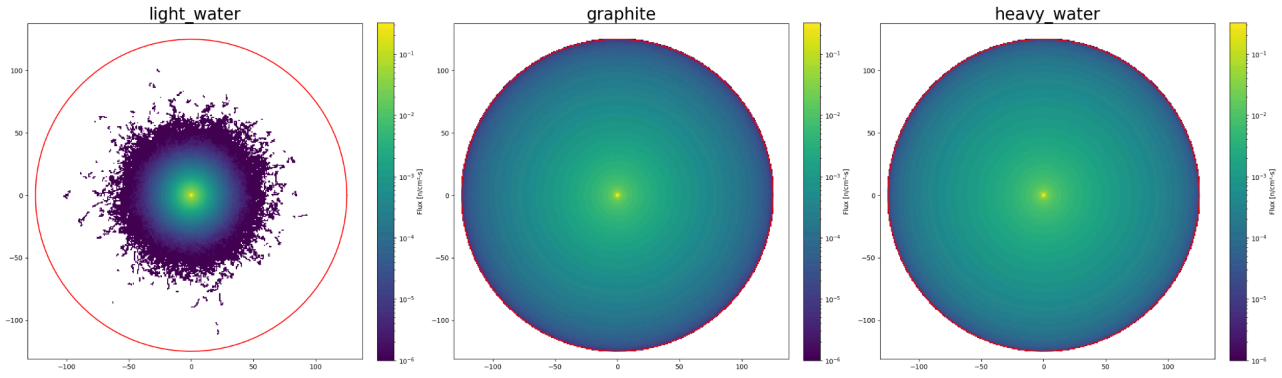


Figure 8. - 2D spatial map of neutron flux distribution

The 2D flux maps of and Graphite exhibit a perfectly saturated geometry, reaching a steady state across the entire volume. The sharp boundary indicates that neutrons are escaping the system (leakage) rather than being scattered back, confirming that a 125 cm radius is physically "small" for materials with high diffusion lengths.

11.5. Leakage & economy

The leakage rate quantifies the number of neutrons crossing the outer boundary of the geometry. While it depends heavily on the system's dimensions, it serves as the key metric for evaluating the shielding performance of a specific configuration. This data is critical for determining the required thickness of a shield or for calculating the reflector savings in a reactor core design.

While previous maps and curves illustrate neutron behavior inside the medium, the leakage table offers the final balance sheet of the simulation. It condenses spatial and material data into a single objective number representing the overall effectiveness of the configuration. By calculating the Relative Leakage (%), we normalize the losses against the source strength (1 n/s), allowing a universal comparison regardless of the source intensity.

```
print(f"{'Material':<15} | {'Leakage (%)':<15} | {'Absorption (%)':<15} | {'Total Loss'}")
print("-" * 70)

for name in results_spectrum.keys():
    # 1. Leakage extraction (Current at the surface)
    total_leak = results_leakage[name]
    rel_leak = (total_leak / 1.0) * 100

    # 2. Absorption extraction (Global absorption tally)
    total_abs = results_abs[name]
    rel_abs = (total_abs / 1.0) * 100

    # 3. Conservation balance (Should be close to 1.0)
    total_loss = total_leak + total_abs

    # Formatted output
    print(f"{name:<15} | {rel_leak:<14.2f}% | {rel_abs:<14.2f}% | {total_loss:.4f}")

    # Small warning if the balance does not close (convergence or tally issue)
    if abs(total_loss - 1.0) > 0.01:
        print("\n[WARNING] Neutron balance deviation > 1%. Check statistics or tally
       definitions.")

    print("-" * 70)
```

Material	Leakage (%)	Absorption (%)	Total Loss
H2O	0.00	99.98	0.9998
Graphite	50.09	49.93	1.0002
D2O	90.12	10.39	1.0051

Table. 3 - Global neutron balance

To distinguish between a material that is "opaque" due to scattering or to capture, we analyze the macroscopic absorption rate. With its absorption rate of 99.98%, light water causes a sharp spatial collapse of the neutron flux, preventing the formation of a long-range thermal field due to the high thermal capture cross-section of hydrogen (H_1). In comparison, graphite exhibits a balanced competition between absorption (49.93%) and leakage (50.09%). It allows some neutron migration, its moderate capture cross-section limits its efficiency compared to heavy water. Finally, for heavy water (D_2O), the decrease in flux is not driven by high absorption, but is almost exclusively due to leakage and geometric attenuation.

Let’s recall that this specific radius was chosen through a sensitivity analysis to achieve a leakage rate for graphite which maximizes the separation between the three moderators.

11.6. Quantitative Synthesis

11.6.a - The moderating ratio (M.R.)

The actual efficiency of a moderator is defined not only by its ability to slow down neutrons, but also by its capacity to thermalize them without capturing them. This criterion is the moderating ratio (M.R.) defined by the ratio between the moderating power ($\xi\Sigma_s$ as defined in the "Précis de neutronique"; Paul REUSS) which is the parameter that characterizes the material's ability to slow down neutrons and the macroscopic absorption cross-section (Σ_a):

$$M.R. = \frac{moderating\ power}{\Sigma_a} = \frac{\xi\Sigma_s}{N\sigma_a} \quad (4)$$

Using the data from of the "Précis de neutronique"; Paul. REUSS (Tables 7.4 and 7.1 page 178), we have:

Mat	σ_a(barns)	$N(barns^{-1}.cm^{-1})$	$\Sigma_a(cm^{-1})$	$\xi\Sigma_s$	Moderating ratio	Mean Residence Time (ms)
H2O	0.332 (1xH)	0.03337	0.02216	1.3772	~62	0.15
C	0.00337 (1xC)	0.0802	0.00027	0.0604	~224	3.14
D2O	0.00051 (1xD)	0.0332	0.0000338	0.1795	~5310	4.18

Table. 4 - Nuclear properties

This calculations confirms the results obtained in our simulation :

- For **light water**, despite possessing the highest moderating power (1.3772), its high thermal capture cross-section leads to a significantly low moderating ratio of approximately **62**. This indicates that the probability of a neutron surviving the thermalization process is almost zero, directly confirming the 99.98% absorption and consistent with the 0.15ms of MRT.
- **Graphite** exhibits an intermediate moderating ratio of approximately **224**, which allows for partial neutron survival and results in a balanced competition between leakage (50.09%) and absorption (49.93%).
- Finally, **heavy water** represents the most extreme case; although its moderating power is nearly eight times lower than that of light water, its absorption is almost non-existent. This causes the moderating ratio to explode to approximately **5310**, creating a "transparent" medium that facilitates long-distance diffusion justifying the high leakage rate of 90.12%. Consequently, neutrons reside there much longer, yielding the highest mean residence time, reaching 4.18 ms.

11.6.b - Final discussion on material properties

The numerical results show that H_2O is the most efficient at slowing neutrons down but it is a poor candidate for large-scale thermal fields. At the contrary, D_2O allows neutrons to persist in the system, enabling the "accumulation effect" seen in the temporal plateau and the high leakage values.

For H_2O, the 125 cm radius is effectively "infinite" but for D_2O and graphite, the 125 cm radius is "thin," as evidenced by the high residence time and high leakage, meaning the neutrons have not yet exhausted their diffusion potential.

12. FINAL CONCLUSION AND ENGINEERING PERSPECTIVES

12.1. Industrial implications & reactor design

These results confirm and justify the historical and technological choices of the nuclear industry :

- H_2O : Allows for highly reduced core volumes. However, this compactness requires enriched uranium to compensate for the neutrons "stolen" by hydrogen capture.
- The near-zero capture of D_2O and graphite prioritizes neutron survival. The 90% leakage rate for D_2O proves that natural uranium reactors must be voluminous (several meters) to reach criticality, bypassing the need for enrichment.

The nuclear design thus illustrates a permanent trade-off between the compactness and passive safety of light water and the optimization of neutron economy (enabling the use of natural uranium) for heavy water and graphite, on the other hand.

12.3. Neutronic analysis based on cpu runtime

Material	**CPU runtime (s)**
Light water (H_2O)	993
Graphite	2808
Heavy water (D_2O)	6801

Table. 5 - CPU runtime per material

The observed disparity in CPU runtimes is a direct macroscopic signature of the microscopic physics at play. In a Monte Carlo simulation, the computational cost is proportional to the total number of simulated events (scattering, boundary crossing). To validate quantitatively, we define the complexity factor by the sum of slowing-down collisions (n_{sd}) and thermal diffusion events (n_{th}) :

$$F \approx \frac{(n_{sd}+n_{th})_{material}}{(n_{sd}+n_{th})_{H_2O}} \quad (5)$$

Source from J.R. Lamarsh, "Introduction to Nuclear Engineering" for n_{sd} *and* n_{sd}

Moderator	n_{sd}	n_{th}	**F**
H_2O	≈ 18	≈ 20	**1.0**
C	≈ 115	≈ 1370	≈ 2.8
D_2O	≈ 25	≈ 27,000	≈ 3.5

Table. 6 - Theoretical complexity factor (*F*)

The CPU runtime is not here a benchmark of hardware performance; it serves as a macroscopic validation of neutron economy. The recorded runtimes of 993 s for , 2808 s for graphite (F = 2.82) are highly consistent with the theoretical complexity factors calculated.

However, the 6801 s runtime for heavy water (D_2O) significantly exceeds the initial linear prediction ($F = 3.48$). The system's behavior is fundamentally non-linear : each neutron that escapes absorption generates a cascade of additional interactions. The slight difference between the graphite slope (-0.0667) and the heavy water slope (-0.0629) masks this cumulative effect. Due to near-zero absorption in heavy water, unabsorbed neutrons are forced into a prolonged and intensive thermal random walk, undergoing thousands of additional scattering events throughout the entire volume.

12.4. Complementary analysis

While this iso-distance study has already provided a wealth of valuable information regarding spatial attenuation, absorption, and global neutronic behaviors, we could gather even more specific insights by performing a complementary iso-leakage study to analyze and compare the resulting spectra.

References

[1] ***P. Reuss***, *Précis de neutronique, Collection Génie Atomique, EDP Sciences, 2003.*

[2] ***J. R. Lamarsh and A. J. Baratta***, *Introduction to Nuclear Engineering, 3rd ed., Prentice Hall, 2001.*

[3] ***P. K. Romano, N. E. Horelik, B. R. Herman, A. G. Nelson, B. Forget, and K. Smith***, *"OpenMC: A state-of-the-art Monte Carlo particle transport code," Annals of Nuclear Energy, vol. 82, pp. 90–97, 2015.*

[4] ***OECD/NEA Data Bank***, *JEFF-3.3 Evaluated Data Library, Joint Evaluated Fission and Fusion File, OECD Nuclear Energy Agency, 2017.*

[5] ***S. Glasstone and M. C. Edlund***, *The Elements of Nuclear Reactor Theory, D. Van Nostrand Company, Inc., 1952.*

[6] ***W. M. Stacey***, *Nuclear Reactor Physics, 2nd ed., Wiley-VCH, 2007.*